Firebase PWA Biztonsági Terv

https://gemini.google.com/app/cfec7029bdae94f4

User prompt: Szia! Egy ügyféloldali Firebase PWA-hoz (időpontfoglalás és bérletkezelés, nyilvános regisztrációval) szeretném megtervezni a `cancelBooking` (foglalás lemondása és 1 bérletalkalom jóváírása, ha az időpontig még több mint 24 óra van hátra) funkciót. Kérlek, a Firebase Web & PWA Security Framework v1.1.0 szabályai szerint: 1. Végezd el a kockázati besorolást (Tier és Onboarding modell), és csak a feladathoz szükséges modulokat töltsd be!2. Állíts elő egy géppel validálható `PLANNER_TO_CODER` YAML handoff-szerződést (exkluzív fájlzárolással és kötelező pozitív/negatív tesztekkel)! 3. Mutasd meg a vázlatos Cloud Functions v2 (`onCall`) és `firestore.rules` tervet, figyelve a rate-limit tranzakció és az atomi üzleti + append-only ledger tranzakció szétválasztására!

Response: Örömmel segítek megtervezni ezt a kritikus funkciót a `Firebase Web & PWA Security Framework v1.1.0` és a két ügynökös (Planner-Coder) munkafolyamat szabályai szerint.

1. Kockázati Besorolás és Betöltött Modulok

A keretrendszer alapján elvégeztem a projekt értékelését:

- **Security Tier: L2 (Business / Customer Application).** Mivel a rendszer ügyfelek adatait, időpontfoglalásokat és bérlet-egyenlegeket (kritikus, pénzértékű erőforrásokat) kezel, az L2 szint kötelező. A kliens-oldali közvetlen adatbázis-írás ezekre a gyűjteményekre szigorúan tilos.
- **Onboarding Model: C (Public Signup).** Mivel a regisztráció nyilvános, kötelező az `email_verified` ellenőrzés, az App Check, valamint a szigorú Rate Limiting a spamek és túlterhelés ellen.

A feladathoz aktivált keretrendszer modulok:

- `01_SECURITY_CORE.md` (Alapelvek és Lock/Handoff szabályok)
- `02_FIREBASE_SECURITY.md` (Firestore Rules, Cloud Functions v2)
- `05_API_INTEGRATION_SECURITY.md` (API végpontok, Idempotencia, Zod validáció)

2. Architektúra Vázlat (Functions & Rules)

A `cancelBooking` folyamatot a backendre kell terelni egy dedikált `onCall` (Cloud Functions v2) függvénnyel. A tervezés során a legfontosabb a két különálló tranzakció szétválasztása.

Cloud Functions v2 (`cancelBooking`):

1. **Bemenet- és Auth-ellenőrzés:** Zod `.strict()` validáció (`bookingId`, `idempotencyKey`). Szigorú ellenőrzés, hogy `request.auth` létezik-e és az e-mail címe megerősített-e.
2. **1. Tranzakció – Rate Limiter (`FIREBASE-FN-003`):** Egy **önálló** Firestore tranzakció (`db.runTransaction`), amely ellenőrzi, hogy a felhasználó nem lépte-e túl a lemondási limitet (pl. max 5 lemondás / óra). Ha túllépi, a számlálót (`blockedCount`) növeljük, a tranzakciót **hagyjuk lefutni**, és a `resource-exhausted` hibát csak a tranzakción *kívül* (utána) dobjuk el. Így a limitek elhasználása nem tekeredik vissza (rollback) egy későbbi üzleti hiba esetén.
3. **2. Tranzakció – Atomi Üzleti Logika (`FIREBASE-FN-004`):**
   - **Idempotencia ellenőrzés:** SHA-256 `requestHash` generálása a bemenetből. Megnézzük a `_idempotency/{uid}_{idempotencyKey}` dokumentumot. Ha már létezik egyező hash-sel, visszatérünk a korábbi eredménnyel.
   - **BOLA (BFLA) védelem:** Lekérjük a `booking` dokumentumot. Szigorúan szerveroldalon ellenőrizzük, hogy `booking.userId === request.auth.uid`.
   - **Üzleti szabály:** Kiszámoljuk az időt. Ha az időpontig több mint 24 óra van hátra, a felhasználó `passes` (bérlet) dokumentumában megnöveljük a felhasználható alkalmak számát 1-gyel.
   - **Append-only Ledger:** Létrehozunk egy új bejegyzést a `passLedger` gyűjteményben a jóváírásról (tartalmazza a `delta: 1` és a `balanceAfter` értékeket).
   - **Státusz frissítés & Mentés:** A `booking` státuszát `CANCELLED`-re állítjuk, majd a tranzakció végén elmentjük a sikeres idempotencia bejegyzést is.

Firestore Security Rules Terv: A kliens számára a kritikus gyűjtemények csak olvashatók.

- `bookings`, `passes`, `passLedger`: `allow read: if request.auth != null && resource.data.userId == request.auth.uid;`
- Írás: `allow create, update, delete: if false;` (Mindent a backend végez!)

3. PLANNER_TO_CODER Handoff Contract

Ez a géppel validálható YAML szerződés garantálja, hogy a kódoló ágens (VS Code-ban) pontosan a megbeszélt biztonsági szabályok alapján dolgozzon.

```yaml
schema_version: "1.1.0"
framework_version: "1.1.0"
handoff_type: "PLANNER_TO_CODER"
task_id: "TASK-2026-CANCEL-001"
project_context:
  tier: "L2"
  onboarding_model: "C"
```

```
scope:
  goal: "Implement cancelBooking Cloud Function v2 with rate limiting, idempotent ledger update, and corresponding r
  affected_components:
    - "Functions_v2"
    - "Firestore_Rules"
  allowed_files_to_modify:
    - "functions/src/cancelBooking.ts"
    - "firestore.rules"
    - "tests/cancelBooking.rules.spec.ts"
    - "tests/cancelBooking.admin.spec.ts"
  concurrency_lock: "EXCLUSIVE"
  lock_acquisition_mode: "COORDINATION_RESERVATION"
required_security_modules:
  - id: "01_SECURITY_CORE.md"
    module_version: "1.1.0"
  - id: "02_FIREBASE_SECURITY.md"
    module_version: "1.1.0"
  - id: "05_API_INTEGRATION_SECURITY.md"
    module_version: "1.1.0"
security_invariants:
  - id: "INV-001"
    requirement_id: "FIREBASE-RULES-002"
    rule: "Direct client writes to bookings, passes, and passLedger are prohibited (allow create, update, delete: if
  - id: "INV-002"
    requirement_id: "FIREBASE-FN-003"
    rule: "Rate limiter must execute in a standalone transaction prior to the main business transaction."
  - id: "INV-003"
    requirement_id: "FIREBASE-FN-004"
    rule: "Booking cancellation, pass increment, passLedger append, and idempotency record must execute in a single
authorization_expectations:
  - "User must be authenticated and email_verified == true."
  - "User can only cancel a booking where booking.userId == request.auth.uid."
required_tests:
  positive:
    - "Successful cancellation >24h credits pass and creates ledger entry, idempotent retry returns deduplicated: tr
  negative:
    - "Direct client write attempt to bookings fails (assertFails)"
    - "Cancellation of another user's booking fails (BOLA)"
    - "Cancellation <24h away fails without pass credit"
prohibited_actions:
  - "Never allow client-side mutations on sensitive collections."
  - "Never execute external API calls inside a db.runTransaction callback."
```

4. Kódolói Prompt (VS Code AI asszisztenshez)

Másold be az alábbi, szigorúan egybefüggő angol nyelvű promptot a VS Code Antigravity (vagy Continue) asszisztensének. Ez tartalmazza az implementációs direktívákat.

```
Act as the "Coder" agent implementing the TASK-2026-CANCEL-001 handoff contract based on the Firebase Web & PWA Secu
Target: Firebase Cloud Functions v2 and firestore.rules.

Requirements for firestore.rules:
- Lock down the collections: bookings, passes, passLedger, _idempotency, and _rateLimits.
- allow read: if request.auth != null && resource.data.userId == request.auth.uid
- allow create, update, delete: if false

Requirements for functions/src/cancelBooking.ts:
- Use v2 onCall endpoint. Enforce enforceAppCheck: true.
- Validate input using Zod .strict() (bookingId, idempotencyKey).
- Enforce authentication (request.auth != null) and email_verified.
- Implement two strictly separated transactions (FIREBASE-FN-003 and FIREBASE-FN-004).
- Transaction 1 (Rate Limiting): Standalone runTransaction checking per-UID cancel limit. If exceeded, increment blo
- Transaction 2 (Business & Ledger): Atomic runTransaction.
  - Verify _idempotency/${uid}_${idempotencyKey} with SHA-256 requestHash.
  - Read the booking. Verify booking.userId === request.auth.uid (BOLA defense).
  - Check if the booking time is strictly greater than 24 hours from now. If so, increment the associated pass remai
  - Create an append-only ledger entry in passLedger with delta +1 and calculated balanceAfter.
  - Mark booking as CANCELLED.
  - Write the idempotency record.

Do not use nested markdown code blocks or backticks in your output response. Generate the necessary TypeScript and r
```
